WEBRUM-03: Core Web Vitals

> **Series:** WEBRUM — Web Real User Monitoring | **Notebook:** 3 of 10 | **Created:** March 2026 | **Last Updated:** 10/31/2026

Overview

Core Web Vitals (CWV) are Google's standardized metrics for measuring real-world user experience on the web. They focus on three pillars: loading performance, interactivity, and visual stability. Dynatrace captures these metrics via the RUM JavaScript agent using the browser's PerformanceObserver API, making them available for DQL analysis in Grail.

Table of Contents

1. Understanding Core Web Vitals
2. Largest Contentful Paint (LCP)
3. Interaction to Next Paint (INP)
4. Cumulative Layout Shift (CLS)
5. CWV by Page Group
6. CWV Trends Over Time
7. CWV Scoring Dashboard
8. Summary and Next Steps

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS with Grail enabled
RUM Enabled	Web applications with Core Web Vitals capture enabled
Browser Support	CWV are Chromium-based only (Chrome, Edge); Safari/Firefox have partial support
Permissions	`storage:user.events:read`, `storage:user.sessions:read`, `storage:metrics:read`
Previous Notebook	WEBRUM-01: RUM Fundamentals

1. Understanding Core Web Vitals

Google defines three Core Web Vitals that reflect real user experience:

Metric	Full Name	Measures	Good	Needs Improvement	Poor
LCP	Largest Contentful Paint	Loading performance	≤ 2.5s	2.5s – 4.0s	> 4.0s
INP	Interaction to Next Paint	Interactivity responsiveness	≤ 200ms	200ms – 500ms	> 500ms
CLS	Cumulative Layout Shift	Visual stability	≤ 0.1	0.1 – 0.25	> 0.25

> **Note:** INP replaced FID (First Input Delay) as a Core Web Vital in March 2024. INP measures the latency of *all* interactions during a session, not just the first one.

How Dynatrace Captures CWV

The RUM JavaScript agent uses the browser's `PerformanceObserver` API to capture:

- **LCP** — Observed via `largest-contentful-paint` entry type
- **INP** — Calculated from `event` entry types (click, keypress, pointerdown)
- **CLS** — Accumulated from `layout-shift` entries (excluding user-initiated shifts)

These values are reported as part of the user action data and are available as metrics in Grail.

2. Largest Contentful Paint (LCP)

LCP measures the time from when the user initiates navigation to when the largest content element (image, text block, video) is rendered in the viewport. It answers: **"How quickly does the main content appear?"**

Common LCP Elements

Element Type	Example
`<img>`	Hero image, product photo
`<video>` poster	Video thumbnail
Block-level text	Heading, paragraph
CSS `background-image`	Banner background

Factors That Impact LCP

- Server response time (TTFB)
- Render-blocking resources (CSS, JS)
- Resource load time (images, fonts)
- Client-side rendering delays

```
// Field vocabulary corrected 08/12/2026 — this series targets **New RUM**,
but was written
// against names that are null on New RUM data, so these cells returned
nothing while erroring
// nowhere. Verified against 5,556,127 user.events records (schema 0.24.0,
javascript agent):
//   action.type == "Load"             -&gt; characteristics.has_page_summary
== true
//                                       (was characteristics.classifier —
"not intended for query usage", SD 1.349)
//   action.type                       -&gt; user_action.type
(hard_navigation | soft_navigation | same_view | api)
//   action.name                       -&gt; page.detected_name
//   LCP / CLS                         -&gt;
web_vitals.largest_contentful_paint / web_vitals.cumulative_layout_shift
//                                       (stable — the fields Dynatrace's own
DQL examples use), falling back to
//                                       the experimental lcp.start_time /
cls.value (corrected 10/14/2026)
//   app.name                          -&gt; frontend.name (stable; an array
on user.sessions)
// UNITS DIFFER BETWEEN THE TWO FAMILIES. web_vitals.largest_contentful_paint
is a DURATION, so
// `/ 1ms` turns it into milliseconds; lcp.start_time is a PLAIN NUMBER
already in milliseconds,
// and dividing it by 1ms yields null. The CWV cells therefore read
// coalesce(web_vitals.largest_contentful_paint / 1ms, lcp.start_time) and
// coalesce(web_vitals.cumulative_layout_shift, cls.value), against the
2500/4000 ms and 0.1/0.25 thresholds.
// Why both: on the validation tenant every user event comes from the
synthetic engine, which fills
// lcp.* / cls.* but not web_vitals.* (0 of 243,494 page summaries in 30 days,
10/14/2026). That
// is a property of synthetic-monitor data, not a sign that web_vitals.* is
rarely populated.
// Average LCP across all applications in the last 24 hours
fetch user.events, from:-24h
| filter characteristics.has_page_summary == true
| fieldsAdd lcp_ms = coalesce(web_vitals.largest_contentful_paint / 1ms,
lcp.start_time)
| filter isNotNull(lcp_ms)
| summarize {avg_lcp = avg(lcp_ms),
    p75_lcp = percentile(lcp_ms, 75),
    p95_lcp = percentile(lcp_ms, 95),
    sample_size = count()},
    by:{frontend.name}
| sort avg_lcp desc
```

```
// LCP distribution — classify into Good / Needs Improvement / Poor
//
// Corrected 08/12/2026: `fieldsAdd total = sum(action_count)` put an
AGGREGATION in a row-wise
// stage, which fails with "Aggregations aren't allowed here". A percentage-
of-total needs the
// grand total on every row, which means collapsing to one row, keeping the
per-category rows in an
// array, and expanding back out.
fetch user.events, from:-24h
| filter characteristics.has_page_summary == true
| fieldsAdd lcp_ms = coalesce(web_vitals.largest_contentful_paint / 1ms,
lcp.start_time)
| filter isNotNull(lcp_ms)
| fieldsAdd lcp_category = if(lcp_ms &lt;= 2500, "Good",
    else: if(lcp_ms &lt;= 4000, "Needs Improvement",
    else: "Poor"))
| summarize action_count = count(), by:{lcp_category}
| summarize {rows = collectArray(record(lcp_category, action_count)), total =
sum(action_count)}
| expand rows
| fieldsAdd lcp_category = rows[lcp_category], action_count =
rows[action_count]
| fieldsAdd percentage = round(action_count * 100.0 / total, decimals: 1)
| fieldsRemove rows
| sort action_count desc
```

3. Interaction to Next Paint (INP)

INP measures the time from when a user interacts (click, tap, keypress) to when the browser renders the next frame reflecting that interaction. Unlike FID (which only measured the *first* interaction), INP considers *all* interactions and reports the worst one (approximated by the 98th percentile).

What INP Captures

1. **Input delay** — Time from user interaction to event handler execution
2. **Processing time** — Time to execute event handlers
3. **Presentation delay** — Time from handler completion to next paint

Common INP Bottlenecks

Bottleneck	Symptom	Fix
Long tasks on main thread	High input delay	Break up long JavaScript tasks
Expensive event handlers	High processing time	Debounce, use web workers
Forced layout/reflow	High presentation delay	Batch DOM reads/writes

How INP Appears in New RUM

INP lives on page summary events as two fields:

- **`web_vitals.interaction_to_next_paint`** — the INP value itself, a duration (*"The Interaction to Next Paint (INP) value"*). Compare its p75 against Google's 200 ms / 500 ms thresholds.
- **`inp.status`** — why a value is or is not present. `below_threshold` means *"INP is not reported because the value is below the threshold of 40 milliseconds"*; `not_reported` means *"INP is not reported because no relevant user interaction happened"*; `reported` means a value was captured.

`reported` therefore does **not** mean "slow": a 60 ms interaction is reported and is Good against Google's 200 ms threshold. Score INP on the numeric value, and count `inp.status` on page summaries only — the field also appears on view summaries and user actions, so counting it across all events counts each page several times.

> **Sources:** Navigation-related events — semantic dictionary (DT docs) — *"INP is not reported because the value is below the threshold of 40 milliseconds."*; Monitor web performance with DQL (DT docs) (takes the p75 of `web_vitals.interaction_to_next_paint` on page summaries).

```
// INP corrected 10/05/2026. `web_vitals.interaction_to_next_paint` is the
numeric INP (a
// duration) on page summaries; `inp.status` only says why it is or is not
present:
// `below_threshold` = INP under Dynatrace's 40 ms reporting floor,
`not_reported` = no relevant
// interaction, `reported` = a value was captured — NOT "slow". Score INP
against Google's
// 200 / 500 ms on the value, and count page summaries only (inp.status also
sits on view summaries
// and user actions, which counted each page about three times).
// Schema-verified, not execution-verified: the validation tenant holds only
synthetic-monitor
// data, which never records an interaction, so the INP columns read null
there.
// INP by application — measured coverage, p75 and poor share
fetch user.events, from:-24h
| filter characteristics.has_page_summary == true
| summarize {
    page_views     = count(),
    inp_measured   = countIf(isNotNull(web_vitals.interaction_to_next_paint)),
    p75_inp_ms     = percentile(web_vitals.interaction_to_next_paint, 75) /
1ms,
    inp_poor       = countIf(web_vitals.interaction_to_next_paint &gt; 500ms),
    below_40ms     = countIf(inp.status == "below_threshold"),
    no_interaction = countIf(inp.status == "not_reported")
  }, by:{frontend.name}
| sort page_views desc
```

```
// INP corrected 10/05/2026. `web_vitals.interaction_to_next_paint` is the
numeric INP (a
// duration) on page summaries; `inp.status` only says why it is or is not
present:
// `below_threshold` = INP under Dynatrace's 40 ms reporting floor,
`not_reported` = no relevant
// interaction, `reported` = a value was captured — NOT "slow". Score INP
against Google's
// 200 / 500 ms on the value, and count page summaries only (inp.status also
sits on view summaries
// and user actions, which counted each page about three times).
// Schema-verified, not execution-verified: the validation tenant holds only
synthetic-monitor
// data, which never records an interaction, so the INP columns read null
there.
// INP by page — worst p75 first
fetch user.events, from:-24h
| filter characteristics.has_page_summary == true
| summarize {
    page_views     = count(),
    inp_measured   = countIf(isNotNull(web_vitals.interaction_to_next_paint)),
    p75_inp_ms     = percentile(web_vitals.interaction_to_next_paint, 75) /
1ms,
    inp_poor       = countIf(web_vitals.interaction_to_next_paint &gt; 500ms),
    below_40ms     = countIf(inp.status == "below_threshold"),
    no_interaction = countIf(inp.status == "not_reported")
  }, by:{page.detected_name}
| filter page_views &gt; 10
| sort p75_inp_ms desc
| limit 10
```

4. Cumulative Layout Shift (CLS)

CLS measures unexpected layout shifts during the entire lifespan of a page. A layout shift occurs when a visible element changes position from one rendered frame to the next without user interaction.

Common Causes of CLS

Cause	Example	Fix
Images without dimensions	Image loads and pushes content down	Set `width` and `height` attributes
Dynamically injected content	Cookie banner pushes page down	Reserve space with CSS
Web fonts	FOUT (flash of unstyled text) causes text reflow	Use `font-display: swap` with fallback metrics
Ads/embeds	Third-party content loads late	Use `<iframe>` with fixed dimensions

```
// CLS distribution — classify into Good / Needs Improvement / Poor
fetch user.events, from:-24h
| filter characteristics.has_page_summary == true
| fieldsAdd cls_score = coalesce(web_vitals.cumulative_layout_shift,
cls.value)
| filter isNotNull(cls_score)
| fieldsAdd cls_category = if(cls_score &lt;= 0.1, "Good",
    else: if(cls_score &lt;= 0.25, "Needs Improvement",
    else: "Poor"))
| summarize action_count = count(), by:{cls_category}
```

```
// Worst CLS pages — pages with the most layout shifting
fetch user.events, from:-24h
| filter characteristics.has_page_summary == true
| fieldsAdd cls_score = coalesce(web_vitals.cumulative_layout_shift,
cls.value)
| filter isNotNull(cls_score)
| summarize {avg_cls = avg(cls_score),
    p75_cls = percentile(cls_score, 75),
    action_count = count()},
    by:{page.detected_name}
| filter action_count &gt; 10
| sort p75_cls desc
| limit 10
```

5. CWV by Page Group

Aggregate Core Web Vitals by page to identify which pages need optimization:

```
// INP corrected 10/05/2026: scored on the numeric
`web_vitals.interaction_to_next_paint` p75
// against 200 / 500 ms — `inp.status == "reported"` only means the value
cleared a 40 ms floor.
// A page with no measured value is "No data", not "Poor": a null p75 used to
fall through the
// if-chain into the worst bucket (corrected 10/05/2026).
fetch user.events, from:-24h
| filter characteristics.has_page_summary == true
| fieldsAdd lcp_ms = coalesce(web_vitals.largest_contentful_paint / 1ms,
lcp.start_time),
    cls_score = coalesce(web_vitals.cumulative_layout_shift, cls.value)
| summarize {
    p75_lcp_ms   = percentile(lcp_ms, 75),
    p75_cls      = percentile(cls_score, 75),
    p75_inp_ms   = percentile(web_vitals.interaction_to_next_paint, 75) / 1ms,
    page_views   = count()
  }, by:{page.detected_name}
| filter page_views &gt; 20
| fieldsAdd lcp_status = if(isNull(p75_lcp_ms), "No data", else: if(p75_lcp_ms
&lt;= 2500, "Good", else: if(p75_lcp_ms &lt;= 4000, "NI", else: "Poor"))),
    cls_status = if(isNull(p75_cls), "No data", else: if(p75_cls &lt;= 0.1,
"Good", else: if(p75_cls &lt;= 0.25, "NI", else: "Poor"))),
    inp_status = if(isNull(p75_inp_ms), "No data", else: if(p75_inp_ms &lt;=
200, "Good", else: if(p75_inp_ms &lt;= 500, "NI", else: "Poor")))
| sort page_views desc
| limit 15
```

6. CWV Trends Over Time

Tracking CWV over time helps identify regressions after deployments, seasonal patterns, and the impact of optimizations.

```
// LCP trend over the last 7 days — hourly p75
fetch user.events, from:-7d
| filter characteristics.has_page_summary == true
| fieldsAdd lcp_ms = coalesce(web_vitals.largest_contentful_paint / 1ms,
lcp.start_time)
| filter isNotNull(lcp_ms)
| makeTimeseries p75_lcp = percentile(lcp_ms, 75), interval:1h
```

```
// CLS trend over the last 7 days — daily p75
fetch user.events, from:-7d
| filter characteristics.has_page_summary == true
| fieldsAdd cls_score = coalesce(web_vitals.cumulative_layout_shift,
cls.value)
| filter isNotNull(cls_score)
| makeTimeseries p75_cls = percentile(cls_score, 75), interval:24h
```

7. CWV Scoring Dashboard

Create a single-query CWV scorecard showing, for each metric, the percentage of page loads in each category. Each percentage is taken over the page loads that measured that metric, so a page that never reported INP does not count as passing it:

```
// Corrected 10/05/2026: each "good %" / "poor %" divides by the page
summaries that actually
// MEASURED that metric (lcp_n, cls_n, inp_n) — dividing by every page summary
diluted the pass
// rates with pages that never reported the value. INP is scored on the
numeric
// `web_vitals.interaction_to_next_paint` (≤ 200 ms good, &gt; 500 ms poor),
not on inp.status.
fetch user.events, from:-24h
| filter characteristics.has_page_summary == true
| fieldsAdd lcp_ms = coalesce(web_vitals.largest_contentful_paint / 1ms,
lcp.start_time),
    cls_score = coalesce(web_vitals.cumulative_layout_shift, cls.value)
| summarize {
    total     = count(),
    lcp_n     = countIf(isNotNull(lcp_ms)),
    lcp_good  = countIf(lcp_ms &lt;= 2500),
    lcp_poor  = countIf(lcp_ms &gt; 4000),
    cls_n     = countIf(isNotNull(cls_score)),
    cls_good  = countIf(cls_score &lt;= 0.1),
    cls_poor  = countIf(cls_score &gt; 0.25),
    inp_n     = countIf(isNotNull(web_vitals.interaction_to_next_paint)),
    inp_good  = countIf(web_vitals.interaction_to_next_paint &lt;= 200ms),
    inp_poor  = countIf(web_vitals.interaction_to_next_paint &gt; 500ms)
  }
| fieldsAdd lcp_good_pct = round(lcp_good * 100.0 / lcp_n, decimals: 1),
    lcp_poor_pct = round(lcp_poor * 100.0 / lcp_n, decimals: 1),
    cls_good_pct = round(cls_good * 100.0 / cls_n, decimals: 1),
    cls_poor_pct = round(cls_poor * 100.0 / cls_n, decimals: 1),
    inp_good_pct = round(inp_good * 100.0 / inp_n, decimals: 1),
    inp_poor_pct = round(inp_poor * 100.0 / inp_n, decimals: 1)
```

8. Summary and Next Steps

In this notebook, we covered:

- **Core Web Vitals overview** — LCP, INP, and CLS with Google's thresholds
- **LCP analysis** — Loading performance measurement and distribution
- **INP analysis** — Interactivity measurement replacing FID
- **CLS analysis** — Visual stability scoring and worst pages
- **Page-level breakdown** — CWV per page group for targeted optimization
- **Trend tracking** — CWV over time for regression detection

- **Scoring dashboard** — Unified CWV scorecard query

Next Steps

- **WEBRUM-04: Session Analysis** — User journey mapping and conversion tracking
- **WEBRUM-06: Performance Analysis** — Deeper performance waterfall analysis beyond CWV

References

- Google Core Web Vitals
- Experience Vitals (DT docs)
- INP Documentation
- User events — semantic dictionary (DT docs) — *"Used for internal optimization when storing the data and not intended for query usage."*
- Semantic Dictionary changelog 1.349 (DT docs)

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.